Blockchain Technology (CSE4444)

Assignment 2: Decentralized Lost-and-Found Registry Using Blockchain

Student Name:	Ashrith Varma
Registration Number:	[YOUR REGISTRATION NUMBER HERE]
Course Code & Title:	CSE4444: Blockchain Technology
Assignment Title:	Assignment 2 - Original Decentralized Application (DApp)
Title of Developed DApp:	Decentralized Lost-and-Found Registry (LostFound Ledger)
Smart Contract Name:	LostFoundRegistry.sol (Solidity ^0.8.20)
GitHub Repository Link:	https://github.com/ashrithvarma2222-lgtm/Blockchain
Submission Status:	All 14 Unit Tests Passed • Deployment Ready (Hardhat/Remix/Ganache)

1. Problem Statement & Project Objectives

Centralized lost-and-found management systems in universities and public venues suffer from a severe lack of transparency, vulnerability to physical logbook loss or manipulation, and lack of trust between the finder and claimant. This project develops an original decentralized application (DApp) that leverages Ethereum blockchain immutability to provide cryptographic proof of lost and found item reports, transparent status transitions, and role-based resolution.

Core Objectives:
• Develop a robust Solidity smart contract enforcing data validation, incremental unique record IDs, and modifiers.
• Integrate MetaMask wallet using Ethers.js for secure transaction signing and real-time state synchronization.
• Deliver an intuitive, responsive frontend allowing users to register items, explore records, and mark resolutions.
• Execute rigorous automated unit tests across valid, invalid, and unauthorized inputs to guarantee contract safety.

2. Technology Stack

Layer	Technology Selected	Role in Project
Smart Contracts	Solidity (^0.8.20)	Implements state variables, modifiers, events, and logic.
EVM Runtime	Hardhat / Ganache / Remix VM	Local blockchain execution and gas estimation.
Web3 Library	Ethers.js (v5.7.2)	MetaMask provider abstraction, ABI encoding, and signer calls.
Frontend Interface	HTML5, CSS3, Vanilla JavaScript	Lightweight, zero-framework, responsive client application.
Testing & QA	Hardhat, Chai, Mocha	14 automated unit tests covering all edge cases.

3. Smart Contract Design & Implementation

The smart contract LostFoundRegistry.sol is engineered to strictly satisfy all grading rubric requirements:

• **Data Structures:** ItemStatus enum (Lost, Found, Claimed, Resolved) and Item struct containing id, title, description, location, contactInfo, reporter address, and block timestamp.
• **Constructor:** Initializes contract owner to msg.sender and resets item counter.
• **Validation Checks:** require(bytes(title).length > 0), require(bytes(description).length > 0), and require(bytes(location).length > 0) prevent empty registrations.
• **Access Control Modifiers:** onlyReporterOrOwner(itemId) guarantees that only the original reporter or admin can resolve an item.
• **Events:** Emits ItemRegistered and ItemStatusUpdated with indexed topics for fast blockchain filtering.

4. Testing Methodology & Actual Results

The project incorporates a complete automated testing suite running in Hardhat. All 14 unit test cases executed successfully with a 100% pass rate. The results strictly cover valid inputs, invalid inputs, and unauthorized inputs:

#	Test Description	Input Category	Result
1	Contract deployment & initial owner assignment	Valid State	**PASSED**
2	Register Lost item, increment counter & emit ItemRegistered event	Valid Input	**PASSED**
3	Register Found item, increment counter & emit ItemRegistered event	Valid Input	**PASSED**
4	Retrieve all registered items (getAllItems)	Valid Input	**PASSED**
5	Retrieve items filtered by reporter address	Valid Input	**PASSED**
6	Query summary statistics (getRegistryStats)	Valid Input	**PASSED**
7	Reporter marks own item as Claimed/Resolved	Valid Input	**PASSED**
8	Contract owner marks item as Claimed/Resolved	Valid Input	**PASSED**
9	Unauthorized account tries to resolve item (Reverts with error)	Unauthorized	**PASSED**
10	Attempt to resolve an already resolved item (Reverts)	Invalid State	**PASSED**
11	Registration with empty title string (Reverts)	Invalid Input	**PASSED**
12	Registration with empty description string (Reverts)	Invalid Input	**PASSED**
13	Registration with empty location string (Reverts)	Invalid Input	**PASSED**
14	Query non-existent item ID (0 or out-of-range reverts)	Invalid Input	**PASSED**

5. DApp Workflow & User Interaction Guide

1. **Connect MetaMask:** User visits http://localhost:8080 and clicks 'Connect MetaMask'. The app connects via Ethers.js and displays the active wallet and ETH balance.
2. **Configure Contract Address:** Users deploying via Remix IDE or Ganache can click 'Contract Settings' in the header and paste the contract address.
3. **Register Lost/Found Item:** User fills in the Title, Location, Contact, and Description, then clicks 'Sign & Broadcast Record'. MetaMask prompts for signing and gas payment.
4. **Blockchain Confirmation:** The transaction is mined, emitting the ItemRegistered event. The item immediately appears in the Registry Explorer with its unique block record ID.
5. **Claim & Resolution:** When an item is reunited with its owner, the original reporter clicks 'Mark Claimed / Resolved', updating the on-chain status.

6. Conclusion & Limitations

Conclusion: The LostFound Ledger DApp successfully satisfies all requirements specified in CSE4444 Assignment 2. The smart contract ensures tamper-proof record keeping, verifiable ownership, and automated validation on the Ethereum blockchain. The frontend provides a rich, responsive interface with seamless MetaMask connectivity.

Limitations & Future Enhancements:

• *Decentralized Image Storage:* Currently text metadata is stored on-chain; future versions can store photo attachments via IPFS/Filecoin.

• *Gas Optimization:* For large-scale production, deploying on Ethereum Layer-2 rollups (Arbitrum, Polygon, Optimism) will drastically reduce transaction fees.